

CSE 256 | Lecture 2: Text Classification with Linear Models

Ndapa Nakashole

January 8, 2026

Administrative matters

- ▶ **PA1:** out by Monday on Canvas

Today

- 1 Text Classification: Problem Definition
- 2 Basic data-driven approaches to text classification:
 - k-Nearest Neighbor
 - Linear Classifier

Text Classification

Task: assign a text document to one or more categories from a predefined set

$$Y(\text{I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet.}) = L$$


- Sentiment analysis
- Spam detection
- Topic classification
- Language identification
- Authorship attribution .

Terminology around Number of Classes

- ▶ Binary classification: Two classes
- ▶ Multi-class classification: one out of many labels
- ▶ Multi-label classification: Assign multiple classes to a single instance

A Cancer Conundrum: Too Many Drug Trials, Too Few Patients

Breakthroughs in immunotherapy and a rush to develop profitable new treatments have brought a crush of clinical trials scrambling for patients.

By GINA KOLATA



→ Health

Yankees and Mets Are on Opposite Tracks This Subway Series

As they meet for a four-game series, the Yankees are playing for a postseason spot, and the most the Mets can hope for is to play spoiler.

By FILIP BONDY



→ Sports

~20 classes

A Text Classifier

- ▶ Can't just write down a neat algorithm ... Unlike tasks with clear rules (e.g., sorting numbers) ...
- ▶ Rules fail here: ...
 - ◉ **Semantic Gap**: The computer doesn't know what words mean, they are just symbols, no knowledge of their relationships, sentiment, etc.
 - ◉ **Intra-class variations**: There are many ways to be a particular label (e.g. SPAM email)
 - ◉ **Scalability**: Have to write rules for every class label

Machine Learning: A Data-Driven Approach

- ▶ Text classification lacks a straightforward, one-size-fits-all algorithm. . . .
- ▶ Instead we will use Machine Learning

*Arthur Samuel (1959): Machine learning is the field of study that gives computers the ability to **learn without being explicitly programmed**.*

Machine Learning: Data-Driven Approach

- 1 Collect a **dataset** of example text inputs and their labels
- 2 Use Machine Learning algorithms to **train** a classifier on the training examples
- 3 **Evaluate** the classifier on new text data
 - A good classifier will **generalize** well - i.e., predict unseen examples correctly

```
def train(training_texts, labels):  
    # Machine learning!  
    return model  
  
def predict(model, test_text):  
    # Use model to predict labels
```

Input Representation: Important Design Choice

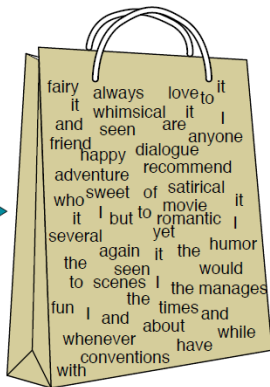
- ▶ Must decide how to represent text before using any ML model
- ▶ ML models accept **numerical input**
- ▶ Given some text, we need to convert it into a numerical feature vector

$$\mathbf{x} = f(\textit{text})$$

Text Representations with Bag of Words (BoW)

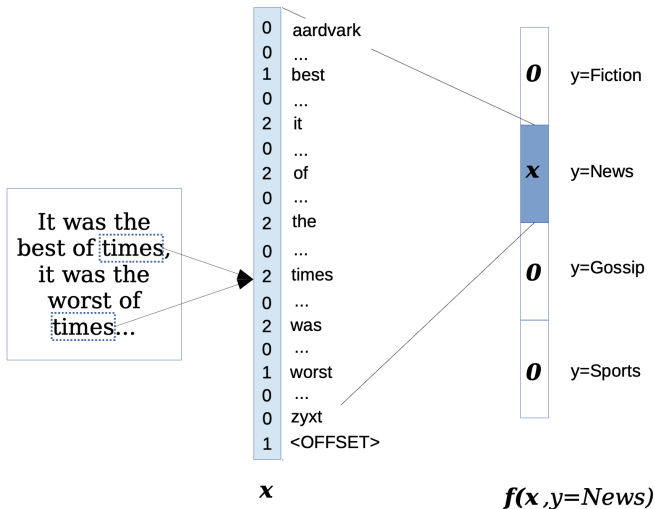
- **Bag of Words (BoW):** Represents text as an unordered collection of words, disregarding grammar and word order

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



Feature Vector: Bow of Words

BoW produces a vector of counts from text - can now use ML!



Each word is a dimension.

Challenges with Bag-of-Words (BoW)

- ▶ **Sparsity:** BoW vectors are high-dimensional and mostly zeros
- ▶ **Loss of sentence structure:** Word order and negation are ignored
 - ⊙ The movie was not good, it was bad
 - ⊙ The movie was not bad, it was good

First classifier: **Nearest Neighbor
Classifier**

First classifier: Nearest Neighbor

```
def train(training_texts, labels):  
    # Machine learning!  
    return model
```



Memorize all
data and labels

```
def predict(model, test_text):  
    # Use model to predict labels  
    return test_label
```

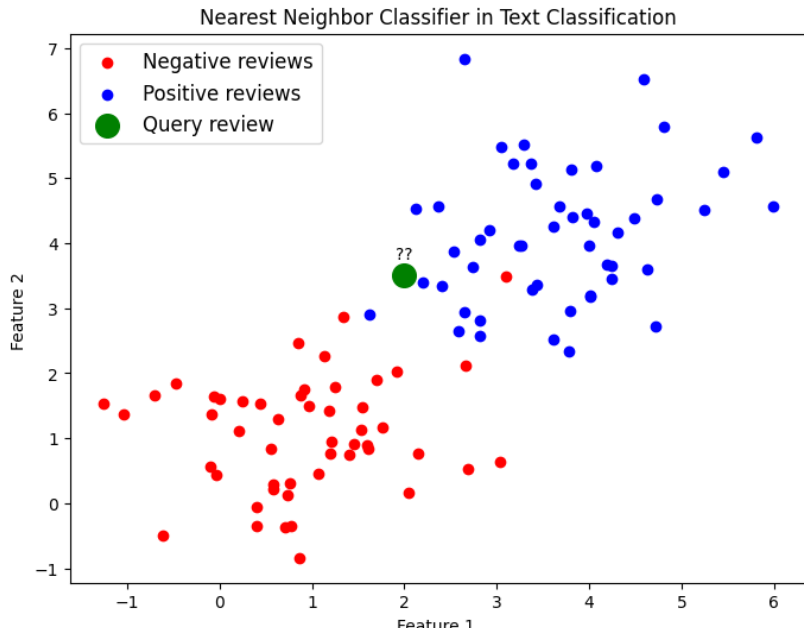


Predict the label of
the most similar
training text
document

L1 distance:

$$d(x, x') = \sum_{i=1}^n |x_i - x'_i|$$

First classifier: Nearest Neighbor



Nearest Neighbor Classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype=self.ytr.dtype)
        # loop over all test rows
        for i in range(num_test):
            # find the nearest training document to the i'th test document
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis=1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
        return Ypred
```

Nearest Neighbor Classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype=self.ytr.dtype)
        # loop over all test rows
        for i in range(num_test):
            # find the nearest training document to the i'th test document
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis=1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
        return Ypred
```

Memorize the
training data

Nearest Neighbor Classifier

```
import numpy as np

class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype=self.ytr.dtype)

        # loop over all test rows
        for i in range(num_test):
            # find the nearest training document to the i'th test document
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis=1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

        return Ypred
```

For each test document:
Find closest train document, predict label of nearest document

Nearest Neighbor Classifier: How does it scale?

```
import numpy as np

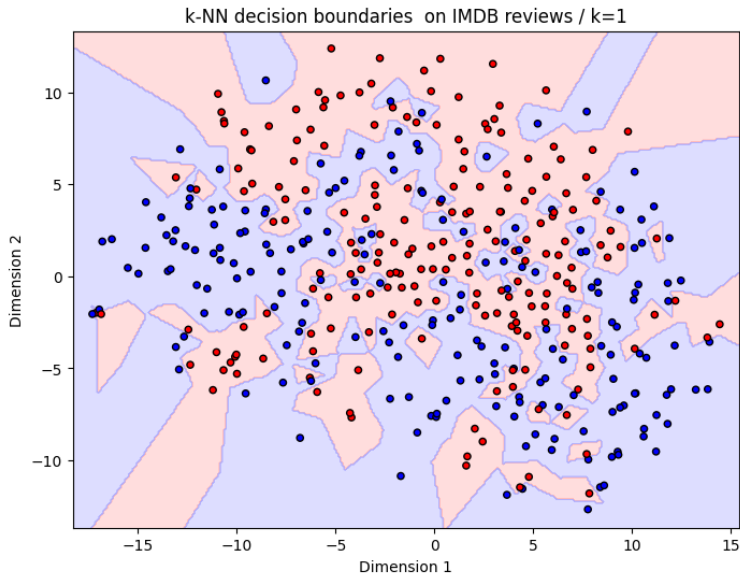
class NearestNeighbor:
    def __init__(self):
        pass

    def train(self, X, y):
        """ X is N x D where each row is an example. Y is 1-dimension of size N """
        # the nearest neighbor classifier simply remembers all the training data
        self.Xtr = X
        self.ytr = y

    def predict(self, X):
        """ X is N x D where each row is an example we wish to predict label for """
        num_test = X.shape[0]
        # lets make sure that the output type matches the input type
        Ypred = np.zeros(num_test, dtype=self.ytr.dtype)
        # loop over all test rows
        for i in range(num_test):
            # find the nearest training document to the i'th test document
            # using the L1 distance (sum of absolute value differences)
            distances = np.sum(np.abs(self.Xtr - X[i,:]), axis=1)
            min_index = np.argmin(distances) # get the index with smallest distance
            Ypred[i] = self.ytr[min_index] # predict the label of the nearest example
        return Ypred
```

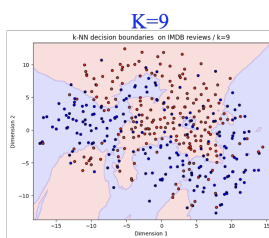
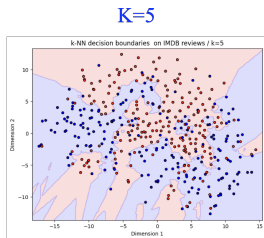
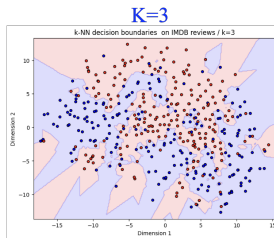
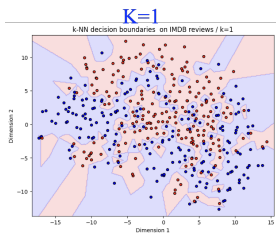
- ▶ Q: How do training and prediction time scale with the number of training examples N ?
- ▶ Ans: Train $O(1)$, predict $O(N)$
- ▶ This is bad: we want classifiers that are **fast** at prediction; **slow** for training is ok

Nearest Neighbor classifier: What does the decision boundary look like?



k-Nearest Neighbors: 1-NN is Unstable, use Majority Vote

Instead of copying label from nearest neighbor, take **majority vote** from k closest points



Hyperparameters

Hyperparameters: external settings of the model

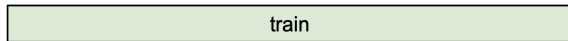
- ▶ What is the best value of k to use?
- ▶ What is the best distance metric to use?
- ▶ What is the best representation to use (Bow, ?, ...)

Note: Hyperparameters are very problem/dataset-dependent

- ▶ There's no universal best choice - must try them all out and see what works best.

Setting Hyperparameters: For any ML model

Idea #1: Choose hyperparameters that work best on the **training data**



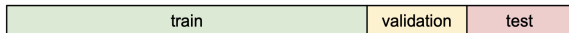
BAD: for k-NN, $K = 1$ always works perfectly if we use training data as test data

Idea #2: Choose hyperparameters that work best on the **test data**



BAD: No idea how algorithm will perform on new data

Idea #3: Split data into **train, validation**; choose hyperparameters on **validation** and **evaluate on test**



Better!

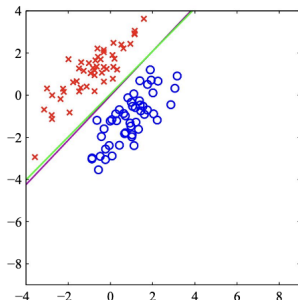
k-NN: Summary

- ▶ The **k-NN classifier** predicts labels based on the k nearest **training examples**
- ▶ Distance metric, k , representation, are **hyperparameters** m
 - ◉ Choose hyperparameters using the **validation set**
- ▶ Golden rule of ML: Only run on the **test set once** at the very end!

Second classifier: Linear Classifier

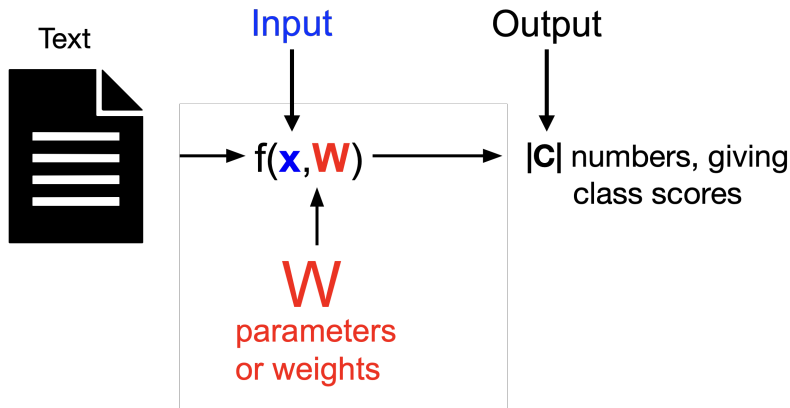
Linear Classifier: a Parametric Approach

- ▶ **Predefined Model Complexity:** assume a specific form for the decision boundary
 - ◉ straight line: defined by a small number of **parameters**: intercept term b and slope w



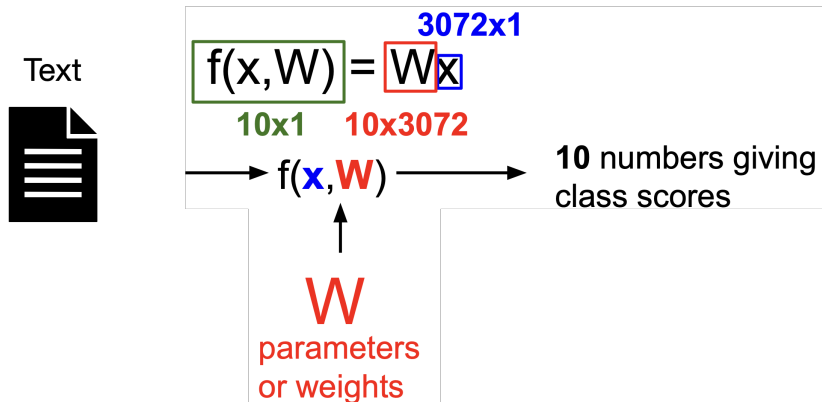
Linear Classifier: notation

A function f of the input features x , and the parameters W :



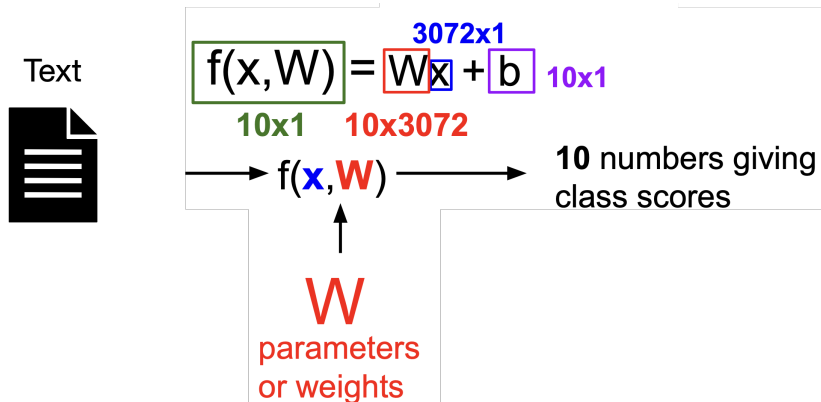
Linear Classifier: example

- ▶ Consider a 10 way classification problem
- ▶ 10 classes, 3072 features
- ▶ $f(x, W) = Wx$



Linear Classifier: bias term

- ▶ Consider a 10 way classification problem
- ▶ 10 classes, 3072 features (assume this is the vocabulary size)
- ▶ Bias term

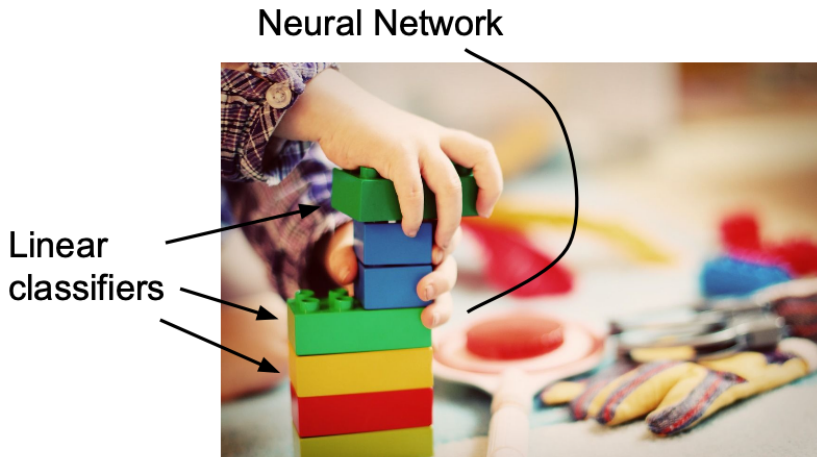


Bias term: intuition

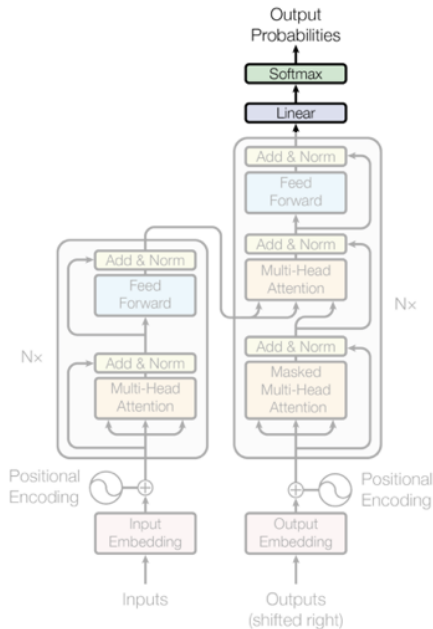
- ▶ Bias term helps tell the model: one class is much more common than others
 - ◉ e.g., consider spam vs non-spam email classification
 - ◉ 90% of emails are non-spam
 - ◉ The model should predict non-spam, unless the content of the email suggests otherwise

Linear Layers: a Building Block in Deep Learning

- ▶ Linear layers: a common building block in neural networks
- ▶ Neural Networks: consist of small functions, like lego blocks, that are easily stacked together to form a complex function



Linear Layers will keep coming back, e.g. in Transformers



Example: Email Classification

Example with a 4-dimensional email representation x , and 3 classes (Spam/Personal/Work)

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

W

3x4

56
231
24
2

x

4x1

+

1.1
3.2
-1.2

b

3x1

=

-96.8
437.9
61.95

Class scores

3x1

first class score: $(0.2 \cdot 56) + (-0.5 \cdot 231) + (0.1 \cdot 24) + (2.0 \cdot 2) = 11.2 - 115.5 + 2.4 + 4.0 = -97.9 + 1.1 = -96.8$

One interpretation: each row of W is a template for a class. The score is the similarity between the input and the template.

Where do the numbers come from?

Diagram illustrating the calculation of Class scores from a weight matrix W and input vector x .

Learned W (3x4):

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

Given x (4x1):

56
231
24
2

Learned b (3x1):

1.1
3.2
-1.2

Calculated Class scores (3x1):

-96.8
437.9
61.95

The calculation is shown as: $Wx + b = \text{Class scores}$

Which W is the best?

Questions ?